

DAY 1 COMPLETE TEXTBOOK & WORKFLOW GUIDE FOR FRIEND B

Embedded Firmware (Circular DMA Ring Buffer in C) & Python Bit-Exact Q1.7 Golden Hardware Model

1. The Big Picture: What Are You Building on Day 1 & How Does It Connect to Friend A?

In Project 1 (AXI4-Stream Tensor Accelerator), Friend A designs the silicon/FPGA hardware multiplier (rtl/q_mult.v) and systolic array in Verilog, while YOU (Friend B) build two mission-critical software components on Day 1:

Deliverable 1 (scripts/golden_model.py): A Bit-Exact Python Hardware Simulator that computes all 65,536 possible 8-bit signed fixed-point (Q1.7) multiplications and outputs a hexadecimal memory file (**sim/golden_vectors.hex**) used to mathematically verify that Friend A's hardware chip has zero bugs.

Deliverable 2 (firmware/dma_ring_buffer.c): A Lock-Free Circular DMA (Direct Memory Access) Ring Buffer in C that allows the microcontroller to stream 1,000 sensor samples/second and 921,600-baud UART debug packets without ever stalling or blocking the CPU.

2. Deep-Dive Theory Part A: Q1.7 Fixed-Point Math & Two's Complement Hex

Why Don't AI Chips Use Standard Float (float32)?

In Python, neural network weights are decimals between -1.0 and +1.0 (e.g., 0.5, -0.25). However, a 32-bit floating-point multiplier requires ~1,500 logic gates on a chip. By converting decimals into 8-bit Signed Integers (INT8, from -128 to +127), the hardware shrinks by 15x in area and uses 10x less energy!

How Does Q1.7 Format Store Decimals Inside an Integer?

In Q1.7 format (1 Sign Bit + 7 Fractional Bits), every decimal number is multiplied by $2^7 = 128$ and stored as an integer:

Real Decimal Value	Formula (Decimal x 128)	8-Bit Signed Integer	8-Bit Binary (Two's Comp)	Hex (a & 0xFF)
+0.50	0.50×128	+64	0100_0000	0x40
+0.25	0.25×128	+32	0010_0000	0x20
0.00	0.00×128	0	0000_0000	0x00
-0.50	-0.50×128	-64	1100_0000 (-128 + 64)	0xC0
-1.00 (Minimum)	-1.00×128	-128	1000_0000 (-128 + 0)	0x80
+0.9921875 (Max)	$+127 / 128$	+127	0111_1111 (0 + 127)	0x7F

The 3 Golden Rules of Q1.7 Multiplication in Your Python Golden Model:

- 16-Bit Raw Multiply:** When you multiply two Q1.7 integers (e.g., $+0.5 \times +0.5 \rightarrow 64 \times 64 = 4096$), the scale factor 128 was multiplied twice (128×128).
- Arithmetic Right Shift by 7 ($\gg 7$):** To cancel out the extra factor of 128, divide by 128 using arithmetic right shift: **shifted_product = (a * b) >> 7**. For $4096 \gg 7$, you get **+32** (which is $32/128 = +0.25$!).
- The -128 x -128 Overflow Trap & Saturation:** When $a = -128$ (-1.0) and $b = -128$ (-1.0), raw product = +16,384, and $16,384 \gg 7 = +128$. Because 8-bit signed numbers only go up to **+127**, +128 overflows! Hardware clamps (saturates) any result > 127 to **+127** and sets **overflow = 1**.

Why Do We Write (a & 0xFF) in Python When Saving Hex Files?

Verilog's \$readmemh cannot read minus signs like '-64'. In Python, bitwise AND with 0xFF (**-64 & 0xFF**) strips the sign extension and gives the exact 8-bit Two's Complement byte (**0xC0** for -64, **0x80** for -128).

3. Deep-Dive Theory Part B: How a Circular DMA Ring Buffer Works in C

Why Standard Arrays Fail in Real-Time Embedded Systems:

When an accelerometer (MPU6050 at 1,000 Hz) or high-speed UART (921,600 baud) streams data, the CPU cannot stop what it is doing to wait for every byte. Instead, a hardware controller called **DMA (Direct Memory Access)** writes incoming bytes directly into RAM in the background while the CPU computes neural network features.

How the Circular Ring Buffer Solves Memory Overflow:

A Ring Buffer connects the end of a fixed C array (index 255) back to index 0 like a circle, managed by two indices:

- **head (Write Index):** Owned by the **DMA Producer**. Every time a new byte arrives, DMA writes to *buffer[head]* and advances *head*.
- **tail (Read Index):** Owned by the **CPU Consumer**. Every time the CPU reads a byte, it reads from *buffer[tail]* and advances *tail*.

Condition / Operation	C Formula (Power-of-2 Size = 256)	Engineering Reason
Buffer is EMPTY	head == tail	The CPU has read every byte that DMA wrote.
Buffer is FULL	((head + 1) & 255) == tail	Advancing head by 1 would collide with unread tail!
Fast Wrap-Around	next_head = (head + 1) & 255;	Bitwise '& 255' takes 1 CPU cycle vs 20+ cycles for '% 256'!
Prevent Compiler Bugs	volatile uint16_t head, tail;	'volatile' forces C to re-read RAM when DMA changes head in an ISR.

4. Annotated Code #1: scripts/golden_model.py

Open **scripts/golden_model.py** in VS Code. Study every line below and run it to generate the 65,536 golden vectors:

```
import os

def q17_mult(a: int, b: int) -> tuple[int, int]:
    """Bit-exact Q1.7 signed fixed-point multiplier matching Friend A's rtl/q_mult.v"""
    # Step 1: 16-bit signed multiplication (range: -16256 to +16384)
    full_product = a * b

    # Step 2: Arithmetic right shift by 7 (divides by 128 while keeping sign)
    shifted_product = full_product >> 7

    # Step 3: Saturation clamping to 8-bit signed range [-128, +127]
    if shifted_product > 127:
        return 127, 1 # Clamp to +127 (0x7F) and set overflow flag = 1
    elif shifted_product < -128:
        return -128, 1 # Clamp to -128 (0x80) and set overflow flag = 1
    else:
        return shifted_product, 0

if __name__ == "__main__":
    os.makedirs("sim", exist_ok=True)
    hex_path = "sim/golden_vectors.hex"
    overflow_count = 0

    with open(hex_path, "w") as f:
        # Sweep all 256 x 256 = 65,536 signed 8-bit input pairs
        for a in range(-128, 128):
            for b in range(-128, 128):
                res, ovf = q17_mult(a, b)
                if ovf:
                    overflow_count += 1
                # Pack {a[7:0], b[7:0], res[7:0], ovf} as Two's Complement Hex
                f.write(f"{a & 0xFF:02X}{b & 0xFF:02X}{res & 0xFF:02X}0{ovf:1X}\n")

    print(f"[Golden Model] Generated 65,536 vectors -> {hex_path}")
    print(f"[Golden Model] Total overflow cases found: {overflow_count} (at a=-128, b=-128)")
```

5. Annotated Code #2: firmware/dma_ring_buffer.c

Open **firmware/dma_ring_buffer.c** in VS Code. This C code implements the non-blocking DMA ring buffer and tests streaming 500 Q1.7 samples across multiple wrap-arounds of the 256-byte buffer:

```
#include <stdio.h>
#include <stdint.h>
#include <stdbool.h>

#define DMA_BUF_SIZE 256 // MUST be a power of 2 (2^8 = 256)
#define DMA_BUF_MASK (DMA_BUF_SIZE - 1) // 255 (0xFF) for 1-cycle bitwise wrap-around

typedef struct {
    int8_t buffer[DMA_BUF_SIZE]; // Stores signed Q1.7 INT8 samples (-128..+127)
    volatile uint16_t head; // Write index (Updated by DMA ISR)
    volatile uint16_t tail; // Read index (Updated by CPU Task)
    uint32_t overflow_drops; // Diagnostic counter for dropped bytes
} dma_ring_buf_t;

void dma_rb_init(dma_ring_buf_t *rb) {
    rb->head = 0;
    rb->tail = 0;
    rb->overflow_drops = 0;
}

// Called by DMA Interrupt when a new byte arrives from UART/SPI/Sensor
bool dma_rb_push(dma_ring_buf_t *rb, int8_t sample) {
    uint16_t next_head = (rb->head + 1) & DMA_BUF_MASK; // Wrap 255 -> 0 using & 255
    if (next_head == rb->tail) {
        rb->overflow_drops++; // Buffer full! Do not overwrite unread CPU data
        return false;
    }
    rb->buffer[rb->head] = sample;
    rb->head = next_head;
    return true;
}

// Called by CPU FreeRTOS Task to process the next available sample
bool dma_rb_pop(dma_ring_buf_t *rb, int8_t *out_sample) {
    if (rb->head == rb->tail) {
        return false; // Buffer empty! CPU returns immediately without blocking
    }
    *out_sample = rb->buffer[rb->tail];
    rb->tail = (rb->tail + 1) & DMA_BUF_MASK;
    return true;
}

int main(void) {
    dma_ring_buf_t uart_dma;
    dma_rb_init(&uart_dma);
    printf("[Embedded Day 1] Testing Lock-Free DMA Circular Ring Buffer...\n");

    // Stream 500 samples (nearly 2x the 256-byte capacity) in 5 bursts of 100
    int total_verified = 0;
    for (int batch = 0; batch < 5; batch++) {
        for (int i = 0; i < 100; i++) {
            int8_t val = (int8_t)((batch * 100 + i) % 127);
            dma_rb_push(&uart_dma, val);
        }
        for (int i = 0; i < 100; i++) {
            int8_t out = 0;
            if (dma_rb_pop(&uart_dma, &out)) total_verified++;
        }
    }
    printf("PASS: Streamed %d Q1.7 samples across circular wrap-around | Drops: %u\n",
        total_verified, uart_dma.overflow_drops);
    return 0;
}
```

6. Step-by-Step Terminal Workflow to Run & Push Day 1

STEP 1: Open VS Code Terminal (Ctrl + ~) inside tensor-edge-accelerator and run Golden Model:
python scripts/golden_model.py

```
# Expected Output:
# [Golden Model] Generated 65,536 vectors -> sim/golden_vectors.hex
# [Golden Model] Total overflow cases found: 1 (at a=-128, b=-128)

# STEP 2: Compile and run the C Circular DMA Ring Buffer:
gcc firmware/dma_ring_buffer.c -o sim/dma_test.exe
./sim/dma_test.exe
# Expected Output:
# [Embedded Day 1] Testing Lock-Free DMA Circular Ring Buffer...
# PASS: Streamed 500 Q1.7 samples across circular wrap-around | Drops: 0

# STEP 3: Sync your Day 1 work to GitHub so Friend A can see your commit:
git pull origin main
git add scripts/golden_model.py firmware/dma_ring_buffer.c
git commit -m "Day 1 [Friend B]: Verified Python Q1.7 golden model and C DMA ring buffer"
git push origin main
```

7. Quick Self-Check Quiz for Friend B (Verify You Mastered Day 1!)

Q1: In Q1.7 format (scale = 128), what integer represents -0.25, and what is (-0.25 & 0xFF) in hex? (*Answer: $-0.25 \times 128 = -32$; $-32 \& 0xFF = 0xE0$*)

Q2: Why does a 256-byte DMA ring buffer hold a maximum of 255 unread bytes instead of 256? (*Answer: Because if head advanced 1 more slot to equal tail, $head == tail$ would look identical to an EMPTY buffer!*)

Q3: Why must *head* and *tail* be declared **volatile** in C? (*Answer: Because the hardware DMA interrupt updates head asynchronously; without volatile, the C compiler might cache an old value in a CPU register.*)